Five rules for intuitionistic primitive recursive arithmetic

Canonical recursion, context clauses, and one opening operation

Research report prepared for Vladimir Reshetnikov

11 November 2026

Abstract

We give a smaller presentation of full first-order intuitionistic primitive recursive arithmetic by combining the current canonical successor-seeded recursor with one-term equality transport and a clause presentation of the paired binder. Public formulas use only equality and one two-variable binder; five inference schemes suffice. The new calculus is explicitly equivalent to the repository's six-rule canonical calculus, with both formula composites proved and numerical translations equal to the identity. Its ordinary realization is therefore equiconsistent and bi-interpretable with iPRA on the whole numerical domain, with actual equality and composite graph $x = y$.

A complete, strictly validated standalone Python checker occupies 3,305 normalized bytes, 946 significant tokens, and 1,072 AST nodes. A control implementing the six-rule calculus with the same variable representation occupies 3,820 bytes, 1,107 tokens, and 1,266 nodes. Gzip level 9 sizes are 1,073 and 1,136 bytes. The simplification is a composition of established mathematical ingredients, coupled to an unambiguous tree grammar; it is not a new arithmetic breakthrough or a claim of absolute minimality. The smaller historical compressed scalar checkers remain smaller in raw source size.

Contents

1 Result and comparison target **2**

2 Five inference patterns with closure clauses **2**

3 The ordinary theory and the iPRA interpretations **6**

4 Scope and substitution audit of the untagged representation **9**

5 Executable implementation and measurements **11**

6 Limits and further research questions **13**

A The complete primary verifier **14**

B Source map and inherited results **16**

1 Result and comparison target

The current repository already contains a six-rule calculus with canonical successor-seeded terms. It also contains the mathematical ingredients needed to make its logical interface smaller: clauses as reusable assumptions, and transport in a single term template. The construction here puts those ingredients together and removes the source restriction on transport, which has no purpose in the straightforward verifier. It is a modest mathematical simplification, with a substantial improvement in the selected transparent implementation.

The exact change is

	canonical repository calculus $\mathcal{K}_6$	this calculus $\mathcal{K}_5$
terms	$x, 0, R$	$x, 0, R$
public formulas	$=, Q, D$	$=, D$
retained context	formulas	clauses $[A; y.B]$
transport input	equation template	one term template
primitive schemes	6	5

The context change is part of the specification: a clause retains both an antecedent and a one-variable consequent. The cost of storing and opening it is counted. The ordinary hypothesis rule and closure specialization become one clause-use rule. The designated contradiction remains $0 = S0$.

Theorem 1.1 (Main result)**.** *The calculus $\mathcal{K}_5$ specified in Section 2 is equiconsistent and bi-interpretable with full first-order* iPRA*. Its numerical domain is the whole domain, interpreted equality is actual equality, and the numerical isomorphism in each composite has graph $x = y$.*

The new part of the proof is an explicit equivalence with $\mathcal{K}_6$. It changes no numerical term or arithmetic operation. The inherited canonical-recursor bridge then gives the displayed result by composition. The proof is written mathematics; this report does not claim a new Lean or Rocq verification.

Throughout, iPRA means intuitionistic first-order arithmetic with a function symbol for each finite primitive-recursive description, its defining equations, zero/successor separation, successor injectivity, and induction for quantifier-free formulas with parameters. It does not mean just the equational theorem fragment. In particular, preserving full intuitionistic logical structure and both interpretation composites is essential.

1.1 What is being minimized

The primary practical comparison is the complete acceptance program: parsing, public syntax and scope validation, canonical substitution, every inference, and final conclusion comparison. Source size is measured after docstring removal and CPython 3.12.14 `ast.unparse` normalization. Token and AST counts are also given. Gzip level 9 is informational, as requested in the earlier project conventions.

There is no proved globally simplest axiomatization. Nor does this report beat every compressed checker in the repository. Its precise positive claims are a smaller interface, a complete equivalence proof, and a smaller matched transparent verifier. An implementation change alone is not described as a new arithmetic axiom deletion.

2 Five inference patterns with closure clauses

This construction combines three already established ingredients: the canonical successor-seeded recursor, one-term crossed transport, and the older clause presentation of the paired binder. The

combination removes the first-class Q constructor and merges its specialization rule with ordinary assumption use. Its novelty is the resulting complete interface and comparison; none of those three ingredients is claimed as new.

2.1 Syntax and every primitive rule

Numerical terms are canonical finite expressions

$$t ::= x \mid 0 \mid R(n, a; u.h), \qquad R(0, a; u.h) \equiv R(0, a; u.0).$$

Only the displayed u in h is bound. Put $Sa := R(0, a; u.0)$. Every substitution is capture-avoiding substitution followed by the canonicalization that erases the body of a literal zero-counter recursor. Thus all occurrence tests below concern canonical syntax. The formula grammar is only

$$F ::= (s = t) \mid D_{x,y}F.$$

A reusable context is a finite list of clauses

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

Clauses are context objects, not formulas or public goals. Their ordinary reading is $A \to \forall y B(y)$. Let

$$J(a) := R(S0, 0; u.a), \qquad c := J(0), \qquad H := (c = 0).$$

The update variable in $J(a)$ is absent from a. A retained ordinary assumption F means the constant clause $[H; v.F]$ with v unused. This abbreviation applies also to the local assumption in the following introduction rule.

There are exactly the following five primitive inference patterns.

1. Computation, in any context:

$$R(Sn, a; u.h) = h\langle R(n, a; u.h)/u\rangle. \tag{R}$$

2. One-term crossed transport, with unrestricted source:

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}. \tag{T}$$

   Here h is a single fixed term template; constant templates are allowed.

3. Clause use:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } [A; y.B(y)] \in \Gamma. \tag{U}$$

4. Paired elimination:

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, [B(b, 0); y.B(b, y)] \vdash F}{\Gamma \vdash F}. \tag{E}$$

   The parameter b is fresh for the retained context, conclusion, and fixed parameters of B.

5. Witness-directed introduction:

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

   The parameter z is fresh for the retained context, witness t, and fixed parameters of B.

All displayed instances use normalized substitution. There is no separate hypothesis rule, zero computation rule, Q formula constructor, Q elimination rule, reflexivity axiom, numerical case rule, or induction rule. These statements describe the interface, not an uncharged saving: the clause pair, its formal parameter, and its substitution are part of the implementation.

2.2 The elementary derivations

Computation proves $J(a) = a$ uniformly in every context, and in particular proves H. Ordinary assumption use is therefore computation of H followed by U on the constant clause. Weakening, fresh renaming, and ordinary-assumption cut follow by induction on derivations. In a cut, each use of the distinguished constant clause is replaced by the supplied proof of its constant conclusion. No general numerical-substitution theorem for the syntax-directed I rule is presupposed here.

Lemma 2.1 (Equality from one term template)**.** *The five-rule calculus derives reflexivity, symmetry, transitivity, numerical congruence, and transport in every literal-equation template.*

Proof. Use $J(a) = a$ as both premises of T with the identity template to get $a = a$. To reverse a supplied $s = t$, use H as major premise and the constant template $h(v) = s$; T returns $t = s$. Given $s = t$ and $t = r$, reverse the former and apply T with identity template to major $t = r$ and minor $t = s$, obtaining $s = r$. Congruence uses major $s = t$ and reflexivity of $h(s)$.

For $E(v) \equiv a(v) = b(v)$, the two applications

$$s = t,\ a(s) = b(s) \implies b(s) = a(t), \qquad s = t,\ b(s) = a(t) \implies a(t) = b(t)$$

give arbitrary equational transport. Each term template is fixed finite syntax; no function variable or function extensionality is introduced. □

Lemma 2.2 (All case introductions)**.** *For every formula B and every term t, the case rule*

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}$$

is admissible.

Proof. The only nonprimitive instance has an equation matrix E and literal zero witness. Use the nonliteral term $c = J(0)$ instead. Under $E(c,0)$, the theorem $c = 0$ and equational transport give $E(0,0)$. Insert the supplied branch by ordinary-assumption cut, obtaining $E(0,Sz)$, and transport back to $E(c,Sz)$. Primitive I at witness c is the case mode and proves the original goal $D_{x,y}E(x,y)$. The goal has not changed; only the witnessing term used by its proof has changed. □

Lemma 2.3 (Transport in every formula)**.** *For every Eq/D template $F(v)$, from $s = t$ and $F(s)$ one can derive $F(t)$, under arbitrary retained clauses.*

Proof. Induct externally on the finite formula template. The equation case was proved above. For $F(v) = D_{x,y}B(v,x,y)$, eliminate $F(s)$ at fresh b, obtaining the clause $[B(s,b,0); y.B(s,b,y)]$. Introduce the target by the derived case rule with witness b. Convert its local $B(t,b,0)$ to $B(s,b,0)$ using symmetry and the induction hypothesis, apply U at Sz, then convert $B(s,b,Sz)$ to $B(t,b,Sz)$ using the induction hypothesis. Close introduction and elimination. Only the strictly smaller matrix is used in the recursive transport, so the construction is not circular. □

2.3 Closure as an abbreviation, not a constructor

For a fresh unused variable w, write

$$q_y B(y) := D_{w,y}B(y).$$

The formula has the following three derived interfaces.

- From $\Gamma, B(0) \vdash B(Sz)$ derive $\Gamma \vdash q_y B$ by the derived case introduction with unused witness.

- From $q_y B$ and $B(0)$ derive $B(t)$: eliminate the paired formula, use the opened clause at t, and close elimination. Its witness is unused.
- In any context containing $[B(0); y.B(y)]$, derive $q_y B$: under its case-introduction assumption $B(0)$ use that clause at Sz.

These are respectively introduction, elimination, and eta expansion of a closure resource. The third is what permits translation of a native DE branch that formerly assumed a first-class Q formula.

For an equation $E(y)$, primitive I at witness zero and unused witness coordinate gives $q_y E$ directly from $\Gamma, E(z) \vdash E(Sz)$. Combining this with the derived closure elimination gives ordinary contextual induction on literal equations. Combining closure introduction with a separately supplied base gives numerical cases for arbitrary native formulas. All progress introductions at a nonzero or nonliteral original witness can also be recovered by the usual unused-coordinate construction: form $q_y E(t, y)$ by progress, then under the base $E(t, 0)$ specialize its closure at Sz and use case introduction at t.

2.4 Exact comparison with the six-rule canonical calculus

Let $\mathcal{K}_6$ be the repository's canonical successor-seeded Eq/Q/D calculus, including its restricted crossed equational transport and root-reversing QE. The canonical-recursors equivalence theorem already gives it ordinary equality transport and ordinary QE as derived rules, and both forms of paired introduction. The following comparison changes neither its term syntax nor its normalization.

Define $T : \mathcal{K}_6 \to \mathcal{K}_5$ by

$$T(s = t) = (s = t), \qquad T(D_{x,y}B) = D_{x,y}T(B), \qquad T(Q_y B) = q_y T(B).$$

Every newly inserted dummy coordinate is fresh and unused. This finite translation preserves free numerical variables, the equation/compound root distinction, and normalized numerical substitution.

Theorem 2.4 (Preservation in both directions)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{K}_6} A \quad \Longrightarrow \quad \mathrm{ord}(T\Gamma) \vdash_{\mathcal{K}_5} T(A).$$

Conversely, for Eq/D formulas Δ, F*,*

$$\mathrm{ord}(\Delta) \vdash_{\mathcal{K}_5} F \quad \Longrightarrow \quad \Delta \vdash_{\mathcal{K}_6} F.$$

Here $\mathrm{ord}(\Delta)$ *replaces each ordinary assumption by its constant clause* $[H; v.F]$*.*

Proof. Forwards, induct on the $\mathcal{K}_6$ derivation. Hyp uses H and the constant clause. Computation is unchanged. Its equation transport and root reversal are supplied by derived equational transport and symmetry. QE becomes the derived q elimination, followed by symmetry when its conclusion is an equation. The unified introduction is preserved exactly: the translation preserves roots and its numerical witness is unchanged.

For DE, the translated premise proves $D_{x,y}T(B)$. Its elimination opens $[T(B)(b, 0); y.T(B)(b, y)]$. Eta expansion derives the translated old branch assumption $q_y T(B)(b, y)$. Ordinary-assumption cut inserts this proof into the inductively translated branch. Close DE. All new dummy variables can be freshened before the construction.

Backwards, track each context occurrence by the rule that introduced it. An ordinary constant clause is represented by its ordinary formula assumption; a clause opened by E is represented by the corresponding $Q_y B(b, y)$ assumption. U on an ordinary clause is Hyp; U on an opened closure is ordinary QE. Computation, T, E, and I are respectively the same computation, derived one-term equality transport, native DE, and the same witness-directed introduction. This proves preservation by induction, including derivations from arbitrary ordinary contexts.

If one also wants a statement for arbitrary externally supplied clauses $[A; y.B]$, interpret each by the already derived ordinary macro $A \to \forall y B(y)$. U is then implication elimination followed by universal elimination. No externally supplied clause is needed by a closed-certificate verifier. □

Theorem 2.5 (Both composites)**.** *The inclusion U of Eq/D formulas into the native Eq/Q/D language satisfies $TU(F) = F$ literally. For every native formula A,*

$$A \dashv\vdash_{\mathcal{K}_6} UT(A).$$

Both numerical translations are identities.

Proof. The literal composite follows immediately from the translation's clauses. For the other composite use the native equivalence

$$Q_y B(y) \dashv\vdash D_{w,y} B(y) \quad (w \text{ unused}).$$

Forwards, under $B(0)$ specialize the supplied Q at Sz and introduce the paired formula by cases. Backwards, DE opens exactly the desired $Q_y B$ at an unused witness, and Hyp closes its branch. Formula congruence then lifts this equivalence recursively through Q and D. Thus external structural induction proves the displayed native composite. No quotient, retract, standard-number restriction, or reinterpretation of numerical equality is introduced. □

Composing this explicit equivalence with the repository's existing canonical-recursor bi-interpretation yields equiconsistency and bi-interpretability with full first-order iPRA. Both numerical domains are whole, equality is actual equality, and both numerical composite graphs remain $x = y$. The designated contradiction $0 = S0$ is unchanged.

2.5 What the smaller interface does and does not establish

The public formula constructors decrease from equality/Q/D to equality/D and the named inference patterns decrease from six to five. The implementation must still store a closure antecedent and a one-bound- variable body in each context entry. Ordinary assumptions require a neutral computation premise, so certificates can become longer even when the verifier becomes shorter. A fair size comparison must count canonical substitution, clause formation and opening, scope validation, input parsing, and the public-goal validator in both programs.

The mathematical equivalence above is a written proof. It is not itself a new Lean formalization, a proof of a global lower bound, or evidence that every historical compact checker has been beaten. Those are separate claims requiring their own measured or formal evidence.

3 The ordinary theory and the iPRA interpretations

3.1 Ordinary realization of the complete calculus

The ordinary reading of the paired binder is

$$(D_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big),$$

and $[A; y.B]^\circ = A^\circ \to \forall y B^\circ(y)$. Equality reads as equality. Each fixed recursor description has a first-order primitive-recursive realization. A bound update variable is a finite-syntax device, not a variable ranging over functions. No universal evaluator for primitive-recursive descriptions is postulated.

R is sound by its defining recursion equation. T is ordinary equality transport followed by symmetry. U is implication elimination followed by universal elimination. E is existential elimination with the stated fresh parameter. In I's case mode, numerical coverage extends the zero and successor cases to every number. In its equation-progress mode, ordinary quantifier-free induction, under the conditional zero instance, gives the required universal equation. Existential introduction supplies the displayed witness. This establishes direct soundness in iPRA, including the nontrivial side conditions.

The normalizing equation

$$R(0, a; u.h) \equiv_{\mathrm{syn}} R(0, a; u.0)$$

is inherited from the canonical representation. In ordinary arithmetic it says that both expressions are Sa. It must be implemented during substitution when a counter becomes literal zero; it has not disappeared from the trusted theory.

3.2 Explicit numerical maps

For each fixed R description, the forward map to ordinary primitive-recursive notation is

$$R(n, a; u.h) \longmapsto I(n, Sa; u.h^{\circ}),$$

where ordinary initialized iteration satisfies $I(0, a; h) = a$ and $I(Sn, a; h) = h(I(n, a; h))$. Variables, zero, and equality are unchanged.

For the other direction, the inherited successor-seeded bootstrap constructs one fixed finite term P and proves internally

$$P0 = 0, \qquad P(Sa) = a.$$

Its complete construction and proof are preserved in `reference/successor_seed_bootstrap.tex`; its finite-case prerequisites are in `reference/successor_seed_prelogic.tex`. These are results of the current proposal, reused here, not new assumptions. Define

$$\widehat{I}(n, a; u.h) := P\big(R(n, a; v.S(h[P(v)/u]))\big)\,.$$

The zero computation is $P(Sa) = a$. The successor computation follows by R and then $P(Sw) = w$. Consequently this single fixed translation recovers initialized scalar iteration, including its independent initial value.

The inherited scalar development supplies a primitive-recursive pairing π with projections L, H_1 satisfying

$$L(\pi(a, b)) = a, \qquad H_1(\pi(a, b)) = b.$$

These are numerical terms on the same domain; the domain is not replaced by their coding range. For a fixed primitive-recursive definition

$$f(0, \vec{p}) = g(\vec{p}), \quad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

its translated term is

$$\begin{aligned} W(v, \vec{p}) &= \pi(SL(v), h^*(L(v), H_1(v), \vec{p})), \\ V(n, \vec{p}) &= \widehat{I}(n, \pi(0, g^*(\vec{p})); v.W(v, \vec{p})), \\ f^*(n, \vec{p}) &= H_1(V(n, \vec{p})). \end{aligned}$$

The zero, successor, projection and composition descriptions translate directly. This is an external recursion on each finite function description. The proof of its defining equations uses internal equation induction with all passive parameters fixed.

3.3 Full logical expressiveness

Let $C(n,a,b) = \widehat{I}(n,a;u.b)$. Then $C(0,a,b) = a$ and $C(Sn,a,b) = b$. The inherited clause construction [3] defines a formula gate G_vA and formula selector $M_v(A,B)$, all as finite Eq/D formulas, with

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \qquad M_{Sv}(A,B) \dashv\vdash B.$$

For equations, gate each side with $C(v,0,-)$ and select their two sides with C. For two D formulas, recurse through aligned fresh binders. If just one selected formula is a D formula, pad the equation with its gate under an unused D before aligning the binders. Thus the recursion terminates on the maximum binder depth. The padding equivalence is $A \dashv\vdash D_{w,y}G_yA$.

Put $p(z) = \widehat{I}(z,0;u.C(u,1,0))$, so $p(1) = 1$ and $p(2) = 0$. The full logical macros are

$$\begin{aligned}
A \Rightarrow_* B &= D_{w,z}M_z(A,B),\\
A \wedge_* B &= D_{w,z}G_zM_{p(z)}(A,B),\\
\exists_* x\, A(x) &= D_{x,y}G_yA(x),\\
A \vee_* B &= \exists_* x\, M_x(A,B),\\
\forall_* x\, A(x) &= A(0) \wedge_* D_{w,z}G_zA(z),\\
\bot_* &= (0 = S0).
\end{aligned}$$

All newly bound variables are fresh and w is unused. These are abbreviations, not extra primitives in the verifier. Their usual intuitionistic introduction and elimination rules follow from the derived case rule, clause use, and paired elimination. For example, implication introduction is case introduction using the zero selector as its assumption and the successor selector as its conclusion; implication elimination opens its clause and specializes at 1. Existential elimination specializes the opened gated clause at 1, using the theorem at its zero gate. Conjunction projections use the two instances 1 and 2.

The native logical derivations, arithmetic equality reflection, and quantifier-free induction are included in the frozen references. They are unchanged because the new calculus derives exactly the old Eq/D consequences. Progress remains restricted to a literal equation: allowing it for every compound matrix would introduce full induction and would not be this presentation of iPRA.

3.4 Why these are bi-interpretations

There are two separate round-trip obligations. The new logical translation removes Q by $Q_yB \mapsto D_{w,y}T(B)$; Section 2 proves its round trips internally. On numerical syntax it is the identity.

For the inherited arithmetic maps, compare each fixed operation with its double translation in the external order of finite descriptions. Once earlier operations have been identified, the two trajectories have the same initial value and provably equal updates. Literal-equation induction proves their equality at every internal counter. This also justifies changes of a recursor body: they follow from a proved trajectory comparison, not an uncharged function-extensionality axiom. The same argument covers ordinary primitive recursion after compilation by pairing.

Formula structural induction then lifts the numerical identities and logical macro equivalences to every formula. Hence the numerical relation $x = y$ is internally a total bijection preserving all interpreted operations in each composite. This is stronger than merely knowing that two theories have the same theorems about standard integers, the same consistency strength, or mutual interpretations. Because falsity is preserved and the finite proof translations run both ways, equiconsistency follows as well.

4 Scope and substitution audit of the untagged representation

The concrete representation uses two disjoint classes of variables. The integers $-1, \ldots, -m$ are free numerical parameters at scope m. The positive integers $1, \ldots, d$ are de Bruijn indices for d locally bound numerical variables. Zero is the numerical constant, not a variable. At positive depth d, the syntax is typed as follows:

object	representation	depths of its children
$R(n, a; u.h)$	$[n, a, h]$	$(d, d, d+1)$
$s = t$	$[s, t]$	(d, d)
$D_{x,y}B$	$[B]$	$(d+2)$

An expression may use free parameters only in $[-m, -1]$. In a paired matrix, y is the nearest variable (index 1) and x is next (index 2). A canonical recursor whose counter is literally zero must have body zero. Arrays have no other permitted arities. Exact integer tests exclude booleans; strings, nulls, floating-point numbers, and arbitrary objects are not terms.

4.1 The opening operation and why lifting is unnecessary

Let $O_d(e, v)$ remove the index $d+1$ while preserving the d nearer indices. On an integer, it replaces $d+1$ by v, decrements an integer greater than $d+1$, and leaves every other integer unchanged. On an array it recursively processes its children at the depths in the table. At each resulting recursor node it erases the body if the resulting counter is zero.

The replacement v must be a numerical term well scoped at positive depth zero. It can contain internally bound variables inside its own recursors, but has no positive variable free relative to its root. Therefore inserting v under any surrounding binders needs no lifting: its positive indices already refer exclusively to its own binders, and its negative parameters are names independent of binder depth. This is the essential precondition. Allowing a positive free variable in v without lifting would be incorrect.

Lemma 4.1 (Scope preservation)**.** *If e has free scope m and ambient positive depth $k > d$, and v is a numerical term at positive depth zero and free scope m, then $O_d(e, v)$ has the same sort as e, ambient positive depth $k-1$, and free scope m. The operation introduces no new free parameter beyond those in e, v. It produces canonical syntax when its inputs are canonical.*

Proof. Structural induction on e. The integer cases respectively remove the selected index, lower an outer index, or preserve a nearer index or parameter. A locally closed replacement remains well scoped under any additional ambient binders. The three array cases use exactly their displayed binder increments. Erasing a recursor body only removes occurrences; it cannot introduce a variable or violate a scope bound. □

4.2 Coherence of canonicalization

Let ν recursively canonicalize raw expressions by replacing $R(0, a; u.h)$ by $R(0, a; u.0)$ after normalizing the children. Let O_d^{raw} be the same opening without body erasure. For canonical inputs e, v, the executable opening satisfies the first identity below; the second is the normalization-coherence identity on arbitrary raw inputs:

$$O_d(e, v) = \nu\big(O_d^{\mathrm{raw}}(e, v)\big), \qquad \nu\big(O_d^{\mathrm{raw}}(\nu e, \nu v)\big) = \nu\big(O_d^{\mathrm{raw}}(e, v)\big).$$

Both identities follow by structural induction. A body discarded before opening had a counter already literally zero, which remains zero under opening. A counter that becomes zero only after opening is

discarded on both sides by the outer normalization. Also $\nu(t) = 0$ if and only if t is literally zero: canonicalization does not collapse an entire recursor node to a number. Thus normalizing early cannot conceal a nonzero counter that subsequently becomes a different constructor.

For a matrix with slots $y = 1, x = 2$ and locally closed terms t, w,

$$O_0(O_1(B, t), w) = \nu(B[t/x, w/y]).$$

The two openings may also be performed in the opposite order, opening y first and then the newly nearest x. Their equality follows from the ordinary simultaneous-substitution lemma, since neither replacement contains a free positive index, followed by the displayed normalization coherence. This identifies the executable operation with the mathematical normalized substitution, including nested recursors and nested D.

4.3 Each use in the five-rule verifier

At free scope m, rule inputs and generated objects have these types:

- A public goal is a formula at positive depth zero. A computation counter and initializer are terms at depth zero, while its body is a term at depth one. Its right side is $O_0(h, R(n, a; u.h))$.
- The single transport template is a term at depth one. Both equation endpoints supplied as images are terms at depth zero, so the two template instances are valid openings.
- A context clause is a pair $[A; y.B]$, with A a formula at depth zero and B a formula at depth one. Clause use supplies a locally closed target term t and concludes $O_0(B, t)$ after proving A.
- An ordinary assumption F becomes $[H; v.F]$. Since F has no positive index free at its root, the identical syntax can be viewed under the unused additional binder v; no shift is needed.
- For paired elimination, use the fresh free parameter $b = -(m + 1)$. From a paired matrix B at depth two, form $C = O_1(B, b)$ at depth one and retain the clause $[O_0(C, 0); y.C]$. The branch runs at scope $m + 1$.
- For paired introduction, validate the witness t at the original scope m, form $C = O_1(B, t)$, and use the fresh parameter $z = -(m + 1)$. Its ordinary branch assumption and target are respectively $O_0(C, \eta)$ and $O_0(C, [0, z, 0])$, where $\eta = z$ in the progress mode and zero otherwise. The array $[0, z, 0]$ is the canonical term Sz.

Old contexts need no shifting when the free scope increases: their negative integers retain the same parameter meanings. The newly chosen negative integer is absent from every previously validated input. A branch can use it, but its discharged result cannot leak it: the outer goal and the introduction witness are validated at the old scope, and the branch conclusion must equal the required outer-scope conclusion or the explicitly constructed target. Independent branches may reuse the same next negative integer because their contexts are separate.

Verifier boundary. Externally supplied goals, witnesses, templates, and computation data must be validated at their specified scopes before any opening. Generated clauses inherit their validity from the preceding rules and the scope lemma. In particular, a positive integer must never be accepted as a free numerical witness merely because it is an integer, and an eigenparameter may not appear in an outer goal. A validator which first erases arbitrary malformed zero-counter bodies may define a different accepted encoding; strict canonical validation must reject such bodies before interpreting them.

The arithmetic normalization theorem used here is exactly the inherited canonical-recursor theorem. This representation adds no new computation law, equality principle, induction instance, or numerical object.

Clause pairs are a separate data structure. A retained pair (A, B) has two components like an equality array, but its second component has an extra binder. It is never passed to the generic expression opener as one object. Clause use opens B alone. This separation is necessary for the stated binding invariant.

5 Executable implementation and measurements

5.1 Acceptance boundary

The primary verifier is `code/arity_checker.py`. Its input is one JSON object with exactly the keys `goal` and `proof`. The goal must be a closed canonical Eq/D formula. The verifier starts with no retained clause and free scope zero. It accepts only when the derived conclusion is literally the submitted goal. Success means exit status zero and the line `ACCEPT`; rejected inputs return nonzero and `REJECT`. It uses no evaluated Python certificates, external theorem table, proof search, downloaded module, or hidden solver.

The five instruction tags are `r`, `x`, `u`, `e`, and `i`. The complete field descriptions and scopes are in `FORMAT.md`. The rule tag is merely a certificate instruction; it is not part of the numerical or formula language. All public goals, numerical witnesses, templates, and computation data are validated before they can supply a substitution image. Generated clauses are valid by the derivation invariant. Invalid keys, malformed arrays, floats, booleans, free positive indices, future negative levels, and noncanonical zero-counter bodies reject.

The mathematical calculus permits every finite proof. Python's ordinary recursion, memory, and running-time limits may reject or fail to finish sufficiently large valid certificates. These resource limits are not extra arithmetic axioms and do not establish an arithmetic decision procedure.

5.2 A comparison that separates the two savings

All programs in the following table are complete standalone acceptance modules. The first is a byte-authenticated copy of the current canonical six-rule checker. The second ports that same calculus to the new variable convention and shared opening operation. It retains its Q constructor, restricted two-sided equation transport, and root-reversing QE. The third implements the five-rule construction. Thus the second-to-third comparison controls the variable representation and isolates the rule/formula-interface change.

Complete verifier	Rules	Tokens	AST	Bytes	Gzip 9
Shipped six-rule	6	1,338	1,485	4,611	1,205
Matched six-rule	6	1,107	1,266	3,820	1,136
Five-rule	5	946	1,072	3,305	1,073

The primary checker saves 515 normalized bytes (13.48%), 161 significant tokens (14.54%), and 194 AST nodes (15.32%) against the matched control. Relative to the exact shipped canonical checker it saves 1,306 bytes (28.32%). The latter percentage combines the mathematical-interface change with the simpler concrete variable/constructor representation; it must not all be attributed to deleting one rule.

The byte counts include JSON decoding, error handling, all scope checks, both syntax validators, the normalizing opener, all five or six rules, and final-goal checking. Each checker relies only on the Python standard library. The reference environment is CPython 3.12.14, and gzip uses compression level 9 and timestamp zero.

The repository also ships much more compressed scalar encodings. For example, its current canonical-recursors package reports a separate 516-byte compact guarded scalar checker, while checked-refinement reports a 788-byte scalar Eq/D public-goal checker. The present 3,305-byte program does not beat those sizes. They are different numerical/certificate presentations. No compressed-code frontier claim is made here, and no earlier obstruction report is treated as a universal lower bound.

Late source update. During final preparation the repository advanced to commit `f77135e4aa10`, filing six additional packages. The canonical six-rule comparator above was unchanged. The newly filed `zero_normal/code/zero_six.py` measures 3,095 bytes, 1,013 tokens, 1,075 AST nodes, and 977 gzip bytes under the same normalization. It uses initialized I with an independent successor and accepts validated general sequents. Our program has fewer rules and tokens but more bytes than that distinct presentation. The late package retains separate Hyp/QE and first-class Q; it does not duplicate the five-rule clause interface. The exact source and measurements are preserved in `evidence/late_source_comparison.json`. Thus the positive byte claim is specifically the matched comparison in the table.

5.3 Verification evidence and its scope

The primary arity checker passes 33 positive and 32 negative named certificates in four modes (readable and normalized code, each under ordinary Python and `python-O`), giving 260 expected-outcome CLI checks. All 33 positive certificates also translate back to accepted proofs of their inclusion images in the pinned six-rule verifier. The matched six-rule control passes 12 positive and 18 negative certificates in the same four modes, giving 120 CLI checks.

An independent named-variable oracle agrees with 12,000 generated term/formula opening computations (6,000 for each main checker), plus three targeted opening controls. A separate certificate audit accepts eight examples per checker and rejects 42 malformed/adversarial primary certificates. The examples include actual witness opening, a used induction hypothesis, nested scopes preserving an older eigenparameter, and substitution exposing a zero counter. Additional tagged-stage translators and regression suites are included as diagnostic evidence; their repetitions are not added to these headline counts.

The evidence includes forward proof translations from the pinned six-rule calculus, reverse proof translations replayed through the pinned verifier, independent arithmetic evaluation/opening tests, and deliberate malformed/scope controls. Tests are run under ordinary Python and `python-O`; acceptance does not depend on assertions. Readable and normalized verifier copies are both exercised.

These checks provide executable evidence for the implementation. The mathematical equivalence is proved in the report. Neither finite testing nor a Python proof translator is a proof-assistant verification of every possible input or derivation. The upstream scalar core has Lean results; the new clause representation and canonical recursor are not presented here as new Lean theorems.

5.4 Reproduction

From the extracted archive directory, run

```
python3 reproduce.py
python3 reproduce.py -pdf
```

The first regenerates measurements and executes the supplied checks. The second also assembles the standalone `report.tex` and builds `report.pdf` using three pdfLaTeX passes. The assembled TeX source has the main proof and the verifier listing embedded, so it can also be compiled directly. Reference sources are preserved separately for inspection of the inherited arithmetic bridge.

6 Limits and further research questions

The achievement. The public formula language has one fewer constructor, the primitive inference inventory has one fewer scheme, and the complete verifier is smaller under a controlled comparison. The numerical language and both numerical interpretation composites are unchanged. The five-rule presentation removes a separately represented logical closure; it does not delete primitive-recursive strength.

The source of the improvement. The three ingredients were already present separately in the repository: canonical successor-seeded recursion, clause contexts, and one-term transport. Combining them makes the array arities unambiguous: three children mean a recursor, two an equality, and one a paired formula. This allows one recursive opener. The free-level convention removes whole-context shifting. Negative rigid parameters and shared openers also have antecedents in the repository; no priority is claimed for those techniques. These are explicit representation choices whose implementations are included in the measured program, not additional semantic axioms.

Why not erase the last computation rule as well? Making both recursor computations definitional and then simply deleting R leaves no source of equation theorems. To see the local problem, keep ordinary natural-number meanings for terms but interpret every syntactic equation atom as false. This is a countervaluation for the remaining inference rules; it does not impose first-order logic's built-in equality axioms. Interpret D and context clauses by their conditional-closure readings. T is vacuous, and U/E are valid logical uses. Case I is sound by coverage; equation-progress I has a false antecedent and therefore introduces a true conditional closure. With no R axiom, no equation, including $0 = 0$, follows from the empty context. Adding a reflexivity or conversion seed repairs the loss, but is a replacement rule, not a free deletion. This argument concerns this specific proposed deletion only; it is not a theorem that every iPRA presentation needs five rules.

6.1 Questions worth pursuing

1. **Formalize the exact bridge.** Can the five-rule calculus be embedded in the existing Lean development with the context-occurrence invariant and the Q-eta expansion proved directly? The canonical recursor's existing written equivalence is a separate prerequisite.
2. **Measure proof size as well as verifier size.** Ordinary assumption use now explicitly proves the fixed neutral theorem. Removing first-class Q can add elimination and eta-expansion nodes. What are the optimal proof-DAG translations, and how much sharing avoids repeated neutral premises?
3. **Port the disjoint-arity grammar to a compact validated kernel.** Does the absence of both an independent successor constructor and a Q constructor yield a genuinely smaller fully validated compact program once normalization and every malformed-input case are charged? This report does not answer that question.
4. **Separate a natural presentation from a minimal implementation.** The witness-directed introduction is concise but not syntactically stable under substitution: a substituted witness

can become literal zero. Can a substitution-stable presentation with comparable full-checker size simplify the formal metatheory?

5. **Prove meaningful restricted lower bounds.** One could fix the term grammar, class of decidable syntax operations, and certificate interface before asking whether five schemes can be reduced. A raw count of named rules is otherwise too easy to change by merging cases or moving work into syntax.

6. **Seek a genuinely new arithmetic reduction.** Eliminating the initializer, changing the domain, or replacing the recursor may still be fruitful, but existing self-seeded constructions and finite-range obstructions must be respected. A new candidate must supply both complete interpretation composites, not just standard-model expressiveness.

A The complete primary verifier

Only the module below and Python's standard JSON/runtime facilities are trusted by its command-line interface. The certificate generators, translators, measurements, and tests are independent supporting programs; the verifier does not import them. Source metrics remove the module docstring and normalize the whole remaining module with `ast.unparse`, followed by one line feed. The delivered readable source has the identical acceptance logic.

```
"""Five-rule clausal canonical iPRA, with one opening operation.

Terms: 0, negative free levels, positive bound indices, [n,a,h] (R).
Formulas: [s,t] (equality), [B] (D). Contexts contain (antecedent, body).
Positive indices in a substituted image must all be internally bound.
The input is {'goal': a closed formula, 'proof': a proof tree}.
"""
import json
import sys


def fail():
    raise ValueError('invalid certificate')


def rec(n, a, h):
    return [n, a, 0 if n == 0 else h]


def successor(a):
    return [0, a, 0]


def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return (type(a) is list and len(a) == 3
            and term(a[0], scope, depth) and term(a[1], scope, depth)
            and term(a[2], scope, depth + 1)
            and (a[0] != 0 or a[2] == 0))


def formula(a, scope, depth=0):
    return (type(a) is list
            and ((len(a) == 2 and all(term(t, scope, depth) for t in a))
                 or (len(a) == 1 and formula(a[0], scope, depth + 2))))


def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
```

```
    size = len(a)
    result = [opening(t, value, depth + (2 if size == 1 else int(size == 3 and i == 2)))
              for i, t in enumerate(a)]
    return rec(*result) if size == 3 else result


def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'u' and len(proof) == 4:
        index, witness, premise = proof[1:]
        if (type(index) is not int or not 0 <= index < len(context)
                or not term(witness, scope)):
            fail()
        antecedent, body = context[index]
        if derive(premise, context, scope) != antecedent:
            fail()
        return opening(body, witness)
    if tag == 'r' and len(proof) == 4:
        n, a, h = proof[1:]
        if not (term(n, scope) and term(a, scope) and term(h, scope, 1)):
            fail()
        return [[successor(n), a, h], opening(h, rec(n, a, h))]
    if tag == 'x' and len(proof) == 4:
        template, major, minor = proof[1:]
        if not term(template, scope, 1):
            fail()
        equation = derive(major, context, scope)
        instance = derive(minor, context, scope)
        if (len(equation) != 2 or len(instance) != 2
                or instance[0] != opening(template, equation[0])):
            fail()
        return [instance[1], opening(template, equation[1])]
    if tag in ('e', 'i') and len(proof) == 4:
        goal, data, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        fresh = -scope - 1
        if tag == 'e':
            paired = derive(data, context, scope)
            if len(paired) != 1:
                fail()
            body = opening(paired[0], fresh, 1)
            clause = (opening(body, 0), body)
            target = goal
        else:
            if len(goal) != 1 or not term(data, scope):
                fail()
            body = opening(goal[0], data, 1)
            eta = fresh if len(goal[0]) == 2 and data == 0 else 0
            clause = ([[[0, 0, 0], 0, 0], 0], opening(body, eta))
            target = opening(body, successor(fresh))
        if derive(branch, context + [clause], scope + 1) != target:
            fail()
        return goal
    fail()


def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True
```

```
if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

B Source map and inherited results

The snapshot is ProveIt commit

`11be5aabc6b07262046e2553419bc6b05271d178`.

All repository paths below are relative to `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/`.

1. `README.md`: current inventory, interpretation standard, and formalization status.
2. `Research/canonical_recursors/theory/canonical_R.tex`: canonical substitution coherence, raw/canonical equivalence, six-rule equivalence, and composition with iPRA. The frozen copy is `reference/canonical_R.tex`.
3. `Research/canonical_recursors/theory/successor_seed_prelogic.tex` and `successor_seed_bootstrap.tex`: finite native case repair and the predecessor/initializer recovery. Copies have the same basenames in `reference/`.
4. `Research/certified_zero/reference/inherited_choice_basis.tex`: the inherited clause interface, logical macros, and formula composites.
5. `Research/certified_zero/reference/inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`: the ordinary scalar bridge, equality reflection, quantifier-free induction, pairing, and primitive-recursion compiler. Copies are in `reference/`.
6. `Lean/ConciseIPRA/NativeOneContext.lean` and `NativeMinimal.lean`: upstream one-term transport and the combined witness-directed calculus. They concern the upstream scalar kernel, not a formalization of this report.

The archive preserves these textual references with the upstream MIT No Attribution license. `evidence/source_provenance.json` records local hashes and upstream Git blob identities. The new theorem is the full five-rule interface and its comparison, including the concrete scope-preserving representation. No priority is claimed for any inherited ingredient.

References

[1] ProveIt Contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned repository snapshot above. https://github.com/VladimirReshetnikov/ProveIt/tree/11be5aabc6b07262046e2553419bc6b05271d178/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt Contributors, *Canonical successor-seeded syntax: a six-rule presentation, with its normalization cost*, `Research/canonical_recursors/theory/canonical_R.tex`, same snapshot.

[3] ProveIt Contributors, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, same snapshot.

[4] ProveIt Contributors, `Lean/ConciseIPRA/NativeOneContext.lean`, `NativeMinimal.lean`, and the `one_context` / `one_sided_transport` research packages, same snapshot.

[5] ProveIt Contributors, *A successor-seeded scalar basis with two computation schemas*; *Pairing, primitive recursion, and internal numerical composites*; source locations in the preceding appendix, same snapshot.